U.PORTO

FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

Project

Ivo Simões up202107703

Aprendizagem Computacional II (CC3043)

2023/2024

1. Introduction

1.1. Sound classification

Sound classification is one of the most common tasks when it comes to audio deep learning and deep learning in general, consisting of taking audio samples. learning patterns from them and using that experience/training to predict new, unseen audio. This project will focus on building classifiers that are able to learn and predict sounds from urban environments, using the UrbanSound8K dataset, and comparing their performances.

1.2. UrbanSound8K

UrbanSound8K is a dataset that contains 8732 audio samples of various urban sounds, each belonging to 1 of the 10 different classes (air conditioner, car horn, children playing, dog bark, drilling, engine idling, gun shot, jackhammer, siren, street music), ranging from a few tenths to 4 seconds in duration.

When it comes to class distribution, this dataset is relatively balanced, with 3 classes (siren, car_horn, gunshot) falling short of the 1000 samples trend that the other 7 classes follow.

2. Models

To address the classification problem presented by the aforementioned dataset, we will use two distinct deep learning models: a Multilayer Perceptron (MLP) and a Convolutional Neural Network (CNN).

2.1. Multilayer Perceptron (MLP)

A Multilayer Perceptron (or as it will be mentioned as later, MLP), is a feedforward neural network, composed of 2 or more layers of fully connected neurons, containing parameters, biases and activation functions. This neural network takes an input and, by applying linear and nonlinear transformations to the data, is able to transform it into a probability distribution that represents the probability of the input belonging to each of the output classes.

The MLP learns through the backpropagation algorithm, taking the loss of the network output with respect to the actual output and propagating it backwards by computing the gradients at each neuron.

Although they might experience problems due to the exponential growth of the number of parameters, MLPs are quite fast to train and very versatile, making them usable in a wide range of classification problems, including the one being tackled in this project.

2.2. Convolutional Neural Network (CNN)

Convolutional Neural Networks (commonly referred to as CNNs) are also feedforward neural networks, capable of learning feature engineering by themselves using kernels and preventing vanishing and exploding gradients by using fewer connections (regularization). The kernel, together with stride and padding, is able to reduce the exponential number of parameters a MLP would have to a much smaller number, thus making CNNs regularized networks.

Due to the advantages described above, CNNs are a very good choice for classification problems involving images or any visual elements, making them also a great candidate for audio classification problems since there are many tools to visualize sound.

3. Implementation

This part will contain descriptions for how each step of the classifiers was made, starting at data preprocessing and ending on model evaluation. It is important to note that many of the hyperparameters and steps done were heavily inspired by previous work documented by other authors, which will be mentioned at the end of the report.

3.1. Data preparation

3.1.1. Feature extraction

Most of the time spent on the project was around data preprocessing implementation and the various techniques available to extract features. All the techniques used are shown in the “Preprocessing techniques” section of the notebook. These include:

- **Mel-spectrogram:** These spectrograms utilize the mel scale to report the sound activity at a certain frequency. Mel-spectrograms have demonstrated to be quite useful for extracting features in the past, with many authors choosing to use them to preprocess audio data.

- **Mel-frequency cepstral coefficients (MFCC):** This was the method that proved to be most efficient for feature extraction in this project. Most of the recent audio classification papers refer to this method when extracting audio features, since it has proven to be quite efficient at this task. The MFCCs also produce a kind of spectrogram, which will be useful for image classification later.

- **Short-time Fourier transform (STFT):** Another way to generate a spectrogram is the STFT, which divides the audio sample into shorter intervals and computes the fourier transforms for each interval. This method has proven to be the least efficient during the experimental phase of the project.

3.1.2. Preprocessing

Having now chosen the best method to extract features (MFCCs), the audio data can now be processed into numerical data. The same processed is applied to all audio samples:

1. Resampling: The samples are resampled to a target frequency of 22050Hz. This value was determined through experimentation.

2. Padding: All audio samples will be treated as if they're supposed to be 4 seconds long. This contributes to consistent data shapes after preprocessing. If the audio is less than 4 seconds long, the array will be padded with 0s, as if there isn't any audible sound left. If the audio is longer than 4 seconds, it is trimmed down to exactly 4 seconds. The effect of this step in the models' performance isn't significant.

3. Extract MFCCs: After padding, 128 MFCCs will be extracted, with a FFT window of 1024. These 2 values were obtained from the paper mentioned before.

4. Scaling: Since MFCCs can vary in magnitude, standard scaling will be applied along each MFCC to avoid extreme values in the data. This was proven to be beneficial for the model through experimentation.

5. Mean: The training data for the CNN doesn't follow this step since the network can take a high value of parameters without regularization problems (the final shape being (128, 173)). This step is only performed

for the MLP data to prevent a high number of parameters in the network. To accomplish this, the mean of each MFCC is computed, reducing the input shape from (128, 173) to (128) for each audio sample.

6. Data augmentation (not used): Even though there's an implementation for data augmentation in the notebook, it was not used to obtain the final results since this process was extremely slow. There were a few experiments that suggested that pitch change to samples improved the models' generalization of the data, suggesting a possible improvement in the performance if data augmentation was to be used. Each sample of augmented data follows steps 1-5 of preprocessing.

3.2. Model design

The MLP design consists of 128 input units, followed by 2 hidden layers containing 256 neurons individually, each one using the ReLU activation function. Dropout is applied on both hidden layers to improve the MLP's data generalization, each dropout having a probability of 0.2. The output layer contains 10 neurons representing the 10 classes, with the activation function being Softmax.

The CNN design consists of a convolutional block, which contains three 2D convolutional layers with kernel size 5x5 and stride 1, with two 2D max pooling layers in between with pool size 4x2 and stride 2, and a "linear" layer, where the data is flattened so that it fits the 64 neurons of hidden dense layer, followed by the output layer containing 10 neurons and the Softmax activation function. All convolutional layers utilize the ReLU activation function. Additionally, dropout is used at the last 2 layers, with a probability of 0.2 for the same reason as earlier.

The CNN model was heavily inspired by the paper mentioned previously. The MLP model was built mostly by experimentation.

3.3. Learning strategy

For the loss function and optimizer, cross entropy loss and Adam were used on both models, respectively. The Adam optimizer was chosen since it is one of the most used optimizers and has proven to be a very good optimizer.

As for the training loop, both MLP and CNN were trained using batches of size 100, with the MLP being trained for 100 epochs per fold and the CNN just 25 epochs per fold. The chosen batch size was obtained mostly through experimentation and the number of epochs were chosen based on the time it took to train each model as well as the point where it couldn't learn anymore on the training data.

A type of early stopping was also used to achieve the best test/validation accuracy possible. The epoch where each model would achieve the best validation accuracy was the model instance that was stored for evaluation later on.

4. Results

As it is observable in the section "Comparing and evaluating models", the CNN outperformed the MLP by a margin of around 7% (it is difficult to determine how much better one model was compared to the other since the experiments couldn't be done in a deterministic and reproducible environment).

The MLP was capable of achieving a median accuracy of 61.5%, about 7% lower than the CNN's 68.5%.

Both models achieved their best test accuracy at fold 9, with 69.6% for the MLP and 76.6% for the CNN.

Fold 3 seemed to be the worst for both models too, with the MLP only reaching 50.0% and the CNN getting 63.6%.

5. Comments

In the end, the CNN was able to outperform the MLP at every single fold. This was expected since both models were attempting to perform classification on images generated by the sound data and CNNs are known for performing especially well on image data. It's also worth noting that the training data for both models differs a bit, with the MLP getting more normalized data than the CNN to avoid too many parameters.

If we take a look at the confusion matrices at the end of the notebook, we can also see that both models had difficulty labeling air conditioner, engine idling and jackhammer sounds, with notable confusion between the pairs (engine idling, air conditioner) and (jackhammer, drilling). This confusion can be explained by the similarity between both type of sounds. Further confusion can be observed in examples where sounds are present more in the background than in the foreground (such as children playing, air conditioner and street music).